# Polytech - SAR - Project - 2026

The project is a fault-tolerant totally-ordered multicast.

You will work by group, each group will have four student members, one of them will be the contact member between the group and the professor. The work will be surrendered via the Git repository of the contact member.

In the file `readme.md`, you must specify which branch is to be considered to have the final version of your work. On that branch, the last commit must be before the 8th of November, 2026, before midnight.

# Project Description

The project is a fault-tolerant totally-ordered multicast.

## Messaging & Lamport Logical Clock

Our multicast will be based on the message-oriented platform you developed earlier in the course with `QueueBroker` and `MessageQueue` concepts, implemented over the given circular buffers (rings).

The implementation will therefore be a **fully local implementation** with all tasks running within the same Java Runtime Environment.

The task runtime will be modified to include a **logical clock per task**. The logical clock of a task will tick each time a runnable posted on that task executes.

Also, the task runtime will be modified, if necessary, to ensure a task can be forcefully killed, ensuring that all pending runnables on that task are dropped and will never execute.

## Actors and Groups

Above that platform, you will create a group layer, allowing to constitute a static group of N actors, via a coordinator, called the `group manager`.

You will create a class `Actor` and each actor will have its own private task. An actor may create other private tasks if necessary. **Actors may only communicate through sending and receiving messages**.

Once a group has its N initial actors, it works as a peer-to-peer group, the group manager is not longer in the picture, it is no longer used.

A group has a perfect fault detector watching over all the members of the group. Actors are expected to be fail-stop: either correct or dead. Only actor failures will be considered.

The group offers a listener pattern to know about deaths. Once a member is considered dead, all messages sent to that member will be dropped, all received messages from that member, if any, will be dropped as well.

Although the group is created initially with N actors, it is a dynamic group allowing new actors to join and current members to leave. So the group listener will notify of these two changes of the group as well.

## Simulating Failures

To simulate actor failure, two supports are provided:

- an explicit failure request
- a random failure

The explicit failure request is such that an actor may voluntarily fail, killing all of its tasks.

A random failure is implemented within the task runtime, randomly deciding to kill a task. To help, each task will have a boolean status telling if that task may or may not fail. This is to be able to use some tasks for internal implementation matters without having to handle their failure.

## Totally-ordered Multicast

The design and coding must happen in steps as follows:

- without actor failures
- with actor failures, no reinsertion
- with actor failures, resurrection, and reinsertion.
- fully dynamic group, including actor failure and reinsertion, but also actors leaving and joining voluntarily.

## Testing Scenarii

All these different versions of the totally-ordered multicast must have corresponding test scenarii:

- verifying that all correct actors are delivered the same messages in the same order, with the same content.
- the missed messages by a dead actor are missed forever, even when the actor is resurrected and reinserted in the multicast group.

A special coordinator object , called the `verifier`, will be used per group. That object is not an actor. It will verify the correct behavior of the multicast. To do so, each actor in the group will forward all delivered messages, in the order of delivery. The `verifier` will also be notified of the deaths of actors, their resurrections, and their reinsertions.

All communication with the `verifier` will be done directly, via invoked methods, not via sending and receiving messages, leveraging fully our local implementation (all actors run within the same Java Runtime Environment).

The test scenarii must be able to run in the following two settings:

- voluntary suicide of actors
- random failures of actors

All test scenarii must be run with a variable load, that is,actors must multicast messages via periodic events, with a controllable period. The period can be quite long for early testing and drop to zero in burst mode,

heavily loading the multicast.

## Surrender the Work

The Git repository will be the repository of the contact member, the one he or she used since the start of the SAR course. The final work must be committed in the initial branch "main" or "master".

In the Git repository, there must be a file `readme.md` (markdown format) to help navigate the structure of the Git repository.

- where is the documentation: specification and design documents
- where is the Eclipse project.
- the personal branch for each project member
- the Git name that each member has been using for his or her git-commits
- the "who did what", so what each member worked on.
- how to run the different tests
- reporting on the status of the different tests (pass or fail)

### The Eclipse project

The coding part of the project must be surrendered as an Eclipse project, **NO EXCEPTION**.

You must verify that it can be it imported in a new Eclipse workspace, without requiring any specific steps beside a straight import.

The Eclipse project must be named after the name of the contact member of the student group.

Every member of the group must have a branch, he or she must must have done git-commit and git-merge operations, and must be visible as having contributed via git-blame.

The last commit on the branch "master" must be before the 8th of November, 2026, before midnight.

### Tests

How to run the different tests:

- testing logical clocks
- testing groups
- testing the totally-ordered multicast without actor failures
- testing the totally-ordered multicast with actor failures, no reinsertion
- testing the totally-ordered multicast with actor failures, with resurrection and reinsertion.
- testing the totally-ordered multicast with a dynamic group, including failures and reinsertion, but also actors leaving and joining voluntarily.

When running each test, the messaging period can be set, from a relatively long period to a period of 0ms to have a bust mode.